CampusConnect – Project Synopsis

1. EXECUTIVE SUMMARY

CampusConnect is a web-based platform designed to improve communication and collaboration among students within a college campus.
The system provides a centralized environment where students can connect with each other, share academic resources, and communicate
through real-time messaging.

The platform allows students to create accounts, access a personalized dashboard, upload and download academic notes, and list or
browse books in a student marketplace. In addition, the system includes a real-time campus chat feature that allows students to
communicate instantly with other users.

The application is developed using modern full■stack web technologies. The frontend is built using HTML, Tailwind CSS, and JavaScript
to provide a responsive interface. The backend is developed using Node.js and Express.js, while MongoDB Atlas is used for database
management. Real■time communication is handled using Socket.IO, and file uploads such as PDF notes are managed using Multer.

2. INTRODUCTION

CampusConnect is developed to provide a centralized platform for students to interact, communicate, and share academic resources
within a campus environment.

Currently students rely on multiple platforms like messaging apps or social media to communicate and share materials. These tools are
not designed specifically for academic collaboration. CampusConnect solves this problem by providing a dedicated system where students
can connect, share notes, buy or sell books, and communicate through campus chat.

The system improves collaboration and provides an organized digital environment for campus communication.

3. MODULE DESCRIPTION

Authentication Module
• User registration
• Login authentication
• Secure account access

Dashboard Module
• Central dashboard for accessing features
• Navigation to notes, marketplace, and chat

Notes Sharing Module
• Upload PDF notes
• Download academic materials
• Share resources with students

Marketplace Module
• List books for sale
• Browse books uploaded by other students

Chat Module
• Global campus chat
• Instant real■time messaging using Socket.IO

4. FRONTEND TECHNOLOGY

HTML – Used to create the structure of pages such as login, register, dashboard, notes upload, marketplace, and chat.

Tailwind CSS – Used for styling including responsive layouts, dashboard design, navigation, cards, buttons, and forms.

JavaScript – Used for frontend logic such as sending API requests, loading notes, handling marketplace listings, and managing user interaction.

5. BACKEND TECHNOLOGY

Node.js – Used as the runtime environment to run backend server code.

Express.js – Used to create REST APIs and manage routing.

Example API Routes:
/register
/login
/users
/upload-note
/notes
/sell-book
/books

Multer – Used to upload files such as PDF notes and store them in the uploads folder.

6. DATABASE TECHNOLOGY

MongoDB – Used to store application data such as users, notes, and books.

Collections:
• users
• notes
• books

MongoDB Atlas – Cloud database hosting used to store and manage the project database online.

7. REAL■TIME COMMUNICATION

Socket.IO is used to implement real■time communication in the system. It allows students to send and receive messages instantly
without refreshing the page, enabling a live campus chat environment.

8. SYSTEM ARCHITECTURE

The system follows a full■stack web application architecture:

Frontend (HTML + Tailwind CSS + JavaScript)
↓
REST API (Express.js)
↓
Backend Logic (Node.js)
↓
Database (MongoDB Atlas)

Real■time messaging is handled using Socket.IO between the client and server.

9. SYSTEM REQUIREMENTS

Hardware Requirements
• Processor: Intel i3 or above
• RAM: Minimum 4 GB
• Hard Disk: 500 GB

Software Requirements
• Operating System: Windows / Linux
• Frontend: HTML, Tailwind CSS, JavaScript
• Backend: Node.js, Express.js
• Database: MongoDB Atlas
• Libraries: Socket.IO, Multer
• Tools: Visual Studio Code, npm
• Browser: Google Chrome / Mozilla Firefox